# Requirements Gathering

**Smart Budget Travel & Fare Optimizer**

**Track:** DEPI Data Engineering | **Date:** October 2026
**Team Members:** Yassin Alaa El Din (Lead), Jana Khaled, Peter Fadel, Mohamed Khaled
**Document Status:** Finalized Functional & Technical Requirements

## 1. PURPOSE AND STAKEHOLDERS

This document defines the functional and non-functional requirements for the Smart Budget Travel & Fare Optimizer platform. The system ingests multi-source flight, hotel, and local transit data to provide price forecasts, budget package recommendations, and an automated data quality trust layer. Stakeholders include price-sensitive travelers, business managers, and data engineers.

## 2. PRIMARY DATASETS AND SYSTEM ASSUMPTIONS

The platform utilizes both historical benchmark datasets and periodic web scraping/API ingestion:

- **Historical Data:** Kaggle Flight Prices Prediction Dataset and Hotel Booking Demand Dataset used for initial baseline model training and seasonal price profiling.
- **Live Ingestion:** Dynamic API endpoints (Amadeus, Skyscanner, RapidAPI) and custom scrapers (BeautifulSoup/Selenium/Scrapy) pulling real-time fare updates.
- **Currency & Location:** Standard monetary units in EGP/USD with transparent currency mapping. All price inputs are validated prior to aggregation.
- **Execution Mode:** Orchestrated scheduled batch processing via Apache Airflow operating on defined ingestion intervals.

## 3. DATA PROFILING & KNOWN QUALITY CONSTRAINTS

- **Fluctuating Flight Fares:** Rapid price volatility in dynamic booking APIs require timestamped price snapshot tables to prevent stale recommendations.
- **Missing Accommodation Ratings:** Unrated hotel listings mapped to documented 'unrated' categories without dropping valid room price records.
- **Duplicate Listings across Sources:** Deduplication enforced via composite keys combining provider, departure/check-in time, and location prefix.

## 4. FUNCTIONAL REQUIREMENTS - DATA PIPELINE & ETL

- **FR-1.1 (High):** Extract flight, hotel, and local transportation data from APIs and scrapers into raw staging.
- **FR-1.2 (High):** Clean and standardize date-time fields, monetary values, and missing attributes while retaining immutable raw payloads in MongoDB.
- **FR-1.3 (High):** Load clean data into a dimensional star-schema warehouse in PostgreSQL with explicit Fact and Dimension tables.
- **FR-1.4 (High):** Schedule ingestion workflows using Apache Airflow DAGs with automatic retry mechanisms and detailed task logging.
- **FR-1.5 (Medium):** Implement automated deduplication logic for overlapping travel offers based on provider ID and timestamp.
- **FR-1.6 (High):** Derive consolidated package total costs (Flight + Hotel + Transportation) with explicit handling of taxes and fees.

## 5. FUNCTIONAL REQUIREMENTS - ANALYTICS, AI & RECOMMENDER

- **FR-2.1 (High):** Develop a Time-Series forecasting engine (using Prophet / Scikit-Learn) to output Buy vs. Wait recommendations with confidence scores.
- **FR-2.2 (High):** Implement a Smart Package Recommender that calculates the minimum-cost travel bundle based on user-defined destination and budget constraints.
- **FR-2.3 (High):** Provide an interactive Streamlit dashboard allowing users to filter by travel dates, destination, and maximum budget.
- **FR-2.4 (Medium):** Integrate an AI Assistant supporting natural language questions grounded exclusively in retrieved warehouse data without hallucinating unsupported fares.
- **FR-2.5 (High):** Display historical vs. projected price trend charts with clear date basis and currency indications.

## 6. DATA QUALITY (DQ) & TRUST LAYER REQUIREMENTS

- **FR-3.1 (High):** Detect schema drift, missing mandatory fields, and unexpected data type changes across ingestion sources.
- **FR-3.2 (High):** Identify extreme price anomalies and negative values using rolling z-score baseline rules.
- **FR-3.3 (High):** Assign Warning or Critical severity levels to all detected data quality issues.
- **FR-3.4 (High):** Enforce pipeline quarantine rules: Critical data errors block warehouse loading, while Warnings mark downstream views as degraded.
- **FR-3.5 (High):** Display real-time Data Trust status (Healthy / Warning / Critical) on the interactive dashboard.

## 7. NON-FUNCTIONAL REQUIREMENTS (NFRS)

- **NFR-1 Performance:** Daily batch ingestion and ETL pipeline execution completing under 20 minutes on standard execution environment.
- **NFR-2 Latency:** Dashboard budget filter queries and package calculations responding within 3 seconds.
- **NFR-3 Reliability:** Pipeline error logs recording run IDs, task status, affected row counts, and detailed failure messages.
- **NFR-4 Security:** API keys, database credentials, and secrets stored strictly in uncommitted `.env` files.
- **NFR-5 Reproducibility:** One-command local environment startup using Docker Compose and documented setup guide.